Knowledge Assistant

A searchable, growing memory for documents and notes

The problem

Useful knowledge is easy to lose across PDFs, personal notes, and project records. A one-time document chat can answer questions about a file in front of you, but it does not provide a practical way to build a reusable collection of knowledge and return to it later.

The solution

Knowledge Assistant lets a user upload PDFs and save titled notes, then ask questions across that stored material. It retrieves a limited set of relevant passages and generates an answer grounded in those passages. The response includes source citations so the user can check where supporting information came from.

> Core idea: Save useful information once, add to it over time, and retrieve it later when a question makes that knowledge relevant.

How it works

- Collect: Upload a PDF or save a note with a title and content.
- Prepare: Extract PDF text while preserving page references; split document and note text into smaller chunks.
- Index: Generate embeddings for chunks and store them in PostgreSQL with pgvector.
- Retrieve: Embed the user question, search document and note chunks by vector similarity, and select the most relevant evidence.
- Answer: Send the question and selected passages to Gemini, with instructions to answer from the supplied sources and cite them.
- Verify: Review the answer's source labels and excerpts to trace claims back to a PDF page or saved note.

Architecture

Web interface: Next.js and TypeScript for PDF upload, note saving, question entry, answers, and citations.

API: FastAPI handles document and note endpoints, processing status, and question answering.

Storage and search: PostgreSQL stores documents, notes, chunks, and metadata. pgvector supports similarity search over embeddings.

AI services: Google Gemini creates embeddings and generates answers from retrieved evidence.

Example: interview preparation over time

A user saves a recruiter email as a note, uploads a job description and resume, and adds a project summary PDF. Later, they ask: "Using the interview details in my note, the role requirements, and my project summary, what should I prepare to discuss?"

The system can retrieve relevant passages from the email note and PDFs, connect the information in one response, and cite the supporting sources. The same pattern applies to project research, learning materials, meeting notes, and technical references.

Grounding and privacy considerations

- Answers are instructed to rely only on the retrieved passages and to say when the saved material does not contain enough information.

- Retrieved source text is treated as untrusted content, not as instructions to the model.

- Citations make it easier to inspect the evidence, but they do not guarantee that every answer is correct or complete.

- Users should avoid uploading sensitive information unless the deployment's access controls, retention, and data handling are appropriate for it.

- For a public demo, use fictional or redacted emails and remove personal details from sample documents.

Current scope and next steps

The current application supports PDF and note ingestion, background embedding tasks, semantic retrieval across both source types, and cited question answering. Embedding tasks run as FastAPI in-process background tasks, so a process restart can interrupt work. A durable job queue, database migrations, authentication, and production-grade access controls are potential next steps, not features claimed as complete.

Technology

Python, FastAPI, SQLAlchemy, PostgreSQL, pgvector, Google Gemini, Next.js, and TypeScript.